CycloTrack India — Complete Work Plan

Assigned to: Yuvraj

Primary ownership: Backend architecture, database, authentication, API orchestration, persistence, reports, security and final integration.

Goal: replace demo/stub behavior with a genuinely working, testable, integrated system. The uploaded specification explicitly identifies authentication, analysis/upload, detection, classification, intensity, trajectory, full analysis, history, cyclone data, map, models and reports as missing from the current main backend, and states that the ML directories are scaffolding without trained weights. ■filecite■turn0file0■L668-L695■

Phase 1 — Backend foundation & architecture

- ■ Create a clean FastAPI structure: routers/, schemas/, models/, services/, repositories/, core/config, database/, tests/.
- ■ Move endpoint logic out of main.py into feature routers. Keep /api/health and existing public contracts stable.
- ■ Create centralized configuration for DATABASE_URL, JWT secrets, OpenRouter key, GDACS/external provider settings, upload directory, CORS and environment.
- ■ Add consistent exception handlers, request IDs, structured logging, Pydantic validation and standardized error JSON.
- ■ Create Alembic migrations and a PostgreSQL + PostGIS connection. Add startup/shutdown handling and health checks for API and database.

Phase 2 — Database & authentication

- ■ Design tables for users, refresh tokens/sessions, analyses, uploaded files, cyclone observations, prediction results, shelters, alerts, help requests, missing-person reports, reports and model metadata.
- ■ Add proper foreign keys, indexes and PostGIS geometry fields where spatial queries are needed.
- ■ Implement POST /api/v1/auth/register with password hashing, duplicate-email validation and role handling.
- ■ Implement POST /api/v1/auth/login with JWT access token generation and refresh-token/session management.
- ■ Implement GET /api/v1/auth/me, POST /refresh and POST /logout. Protect private routes with FastAPI dependencies.
- ■ Do not trust a client-supplied role; role assignment must be server-controlled.

Phase 3 — Real analysis/upload pipeline

- ■ Implement multipart upload at POST /api/v1/analysis/upload with MIME/type/size validation, safe filenames, storage and metadata persistence.
- ■ Create an analysis record before processing and return a stable analysis_id.
- ■ Implement POST /api/v1/analysis/run as an actual pipeline job rather than an immediate fake 'processing' response.
- ■ Implement GET /api/v1/analysis/{analysis_id} and /status using database state.
- ■ Add job states such as uploaded, queued, processing, completed, failed and cancelled.
- ■ Prevent duplicate processing and make retries safe/idempotent.

Phase 4 — ML inference integration

- ■ Audit the existing ML scaffolding and define exact input/output contracts for detection, classification, intensity and trajectory.
- ■ Do not invent accuracy values or prediction outputs. If weights are unavailable, return a clearly marked model-unavailable state.
- ■ Integrate real model loading with startup caching, CPU/GPU compatibility, preprocessing and postprocessing.
- ■ Implement POST /api/v1/predictions/detect, /classify, /intensity and /trajectory against stored analysis data.

- Implement POST /api/v1/predictions/full-analysis as an orchestrator that executes the required prediction stages and persists their outputs.
- Store model name/version, inference timestamp, input reference and confidence/uncertainty with every result.
- Add unit tests using deterministic fixtures and integration tests against the API.

Phase 5 — Cyclone, map & external-data backend

- Turn GET /api/v1/cyclones/live into a robust GDACS integration with timeout, retries, response validation, caching and provider-failure handling.
- Persist normalized cyclone observations so historical data can be queried without repeatedly calling the external provider.
- Implement GET /api/v1/cyclones with real filters and pagination.
- Implement GET /api/v1/map returning validated GeoJSON from database data.
- Implement landfall, storm-surge and flood endpoints only when backed by real calculations/models/data; otherwise expose explicit unavailable status rather than demo numbers.
- Implement /data-sources/status from actual provider connectivity/last-success timestamps.

Phase 6 — History, reports & backend quality

- Implement GET /api/v1/history, GET /{id}, DELETE /{id} with ownership/authorization checks and pagination.
- Implement report generation from stored analysis results. Generate real PDF reports and store report metadata.
- Implement GET /api/v1/reports/{report_id}/download with authorization and safe file handling.
- Add automated API tests for every route, including invalid inputs, unauthorized requests, missing records and provider failures.
- Add Docker/dev setup, migration commands, seed script, .env.example and deployment documentation.
- Run an end-to-end demo: register → upload → run → monitor → prediction → map/history → report download.

Yuvraj handoff checklist for Dhanesh

- Publish the finalized OpenAPI schema and database ERD before Dhanesh integrates dependent frontend/ML work.
- Document exact request/response models and error codes for every completed endpoint.
- Tag a stable backend integration commit and provide a test account/seed dataset that contains no real personal emergency data.

Definition of Done

■ No endpoint returns fabricated success data merely to satisfy the frontend.

■ Every endpoint has validation, predictable error responses, logging, and tests.

■ Database writes and reads are real and survive server restarts.

■ Frontend calls the real API and handles loading, empty, error, and success states.

■ Authentication/authorization is enforced on protected resources.

■ ML results clearly distinguish real inference from unavailable/not-trained models.

■ External data providers have timeout, retry, validation, caching/rate-limit protection, and a clear degraded mode.

■ Security-sensitive data such as phone numbers, emergency reports, tokens and uploaded files is protected.

■ All migrations, environment variables, seed data, test commands, and deployment steps are documented.

■ The work is demonstrated end-to-end with reproducible test cases.

Integration rule: coordinate through shared API contracts and database schemas. Do not independently redesign routes already consumed by the frontend. The uploaded specification lists the target route architecture and recommends separating routers, services, PostgreSQL/PostGIS, and ML inference rather than keeping everything in one main.py. ■filecite■turn0file0■L1310-L1419■